

LLAMAFACTORY

25 November 2025 16:29

What is llama factory?

LLaMA Factory ek **all-in-one finetuning engine** hai jo tune LLMs ko **EASY MODE** me train and inference karne ke liye banaya hai.

LLaMA Factory = UI + CLI based tool jisse tu bina coding ke LLM ko finetune kar sakta hai (LoRA, QLoRA, Full FT, DPO sab). **SFT + DPO + RLHF + Reward model** sab same platform me.

UI: LlamaBoard (ya WebUI) — beginner-friendly Web interface: model select karo, dataset add karo, parameters set karo, button dabao, train/export/ chat etc. sab GUI se hota hai.

CLI: agar tu scripting/automation ya advanced use karna chahta hai — to command-line interface available hai: train, export, chat, api serve, etc.

Ye Transformers, PEFT, BitsAndBytes, TRL ko internally wrap karke "one-click" finetuning bana deta hai.

LLMs ko finetune karna super easy ho jata hai (0 coding required).

Finetune model supported:

LLaMA (Meta) & its variants

Mistral (aur uska MoE variant Mixtral-MoE)

Qwen (aur Qwen-series / Qwen-VL / Qwen-2 / etc.)

Gemma models

Yi model family

Phi model family

Baichuan + other community / third-party (e.g. ChatGLM, DeepSeek, etc.)

Finetuning Types Supported

Method	Kya karta hai	LLaMA Factory me?
SFT	Instruction Finetuning	✓
LoRA / QLoRA	Low VRAM fine-tuning	✓
DPO	Direct Preference Optimization	✓
PPO	RLHF	✓
Reward Model	Human preference scoring	✓
RLAIF	AI-feedback RL	✓
Full Fine-Tune	All weights train	✓

Dataset Format supported by llama factory

Factory 3 formats samajhta hai:

Alpaca Format (Instruction Finetuning)

```
[
{
  "instruction": "What is preference alignment?",
  "input": "",
```

```

    "output": "Alignment is making LLM follow human preferences..."
  }
]
Columns:
  • instruction
  • input
  • output

```

ShareGPT Format (Chat finetuning)

Used for role-based chat data.

```

[
  {
    "conversations": [
      {"from": "user", "value": "Hello!"},
      {"from": "assistant", "value": "Hi, how can I help you?"}
    ]
  }
]

```

DPO Format (Preference Training)

```

[
  {
    "prompt": "Explain RLHF.",
    "chosen": "RLHF aligns model with human feedback.",
    "rejected": "I don't know about RLHF."
  }
]

```

Tip: Dataset jitna clean → Tune model utna perfect.

Step-by-Step Finetuning (Web UI)

Base Model Select
 Dataset Load
 Training Configuration
 GPU Optimization
 Start Training
 Evaluation
 Chat With Your Model
 Export Model

CLI Training (YAML-Based)

Command	Purpose / Description
llamafactory-cli train config.yaml	Run a training/fine-tuning session (LoRA, QLoRA, Full FT)
llamafactory-cli chat config.yaml	Launch CLI chat after training/export
llamafactory-cli eval config.yaml	Evaluate model (perplexity/custom eval)
llamafactory-cli export config.yaml	Merge adapters and export final model
llamafactory-cli api config_api.yaml	Start an API server endpoint (OpenAI-style)

llamafactory-cli webui	Launch graphical interface (training, eval, chat, export)
llamafactory-cli webchat	Launch web-based chat UI
llamafactory-cli version	Show installed version

```
Create: train_llama3_qlora.yaml
model_name_or_path: meta-llama/Meta-Llama-3-8B-Instruct
template: llama3
```

```
finetuning_type: lora
dataset: custom
dataset_format: alpaca
train_file: data/train.json
```

```
output_dir: output/llama3-qlora
cutoff_len: 1024
```

```
per_device_train_batch_size: 1
gradient_accumulation_steps: 8
num_train_epochs: 3
```

```
learning_rate: 2e-4
lr_scheduler_type: cosine
warmup_ratio: 0.03
```

```
lora_r: 64
lora_alpha: 16
lora_dropout: 0.05
```

```
quantization_bit: 4
bf16: true
gradient_checkpointing: true
report_to: none
```

```
llamafactory train train_llama3_qlora.yaml
```

Inference with the Trained Model

```
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
```

```
base = "meta-llama/Meta-Llama-3-8B-Instruct"
adapter = "output/llama3-qlora"
```

```
tok = AutoTokenizer.from_pretrained(base)
model = AutoModelForCausalLM.from_pretrained(base)
model = PeftModel.from_pretrained(model, adapter)
```

```
prompt = "Explain Lora in simple terms."
inputs = tok(prompt, return_tensors="pt")
```

```
outputs = model.generate(**inputs, max_new_tokens=200)
print(tok.decode(outputs[0], skip_special_tokens=True))
```

Without merging (adapter)

With Merged Model

What is llama factory?

LlAMA Factory is an all-in-one fine-tuning framework that makes LLM training and inference very simple. It works with hundreds of models and datasets. **LlAMA Factory is a UI and CLI based tool that lets you fine-tune LLMs without writing code.** It internally wraps Transformers, PEFT, BitsAndBytes, and TRL to provide a “one-click” fine-tuning workflow. This makes fine-tuning LLMs extremely easy, with zero coding required.

UI (LlamaBoard / WebUI):

A beginner-friendly web interface where you can select models, upload datasets, set parameters, and run training, export, or chat — all through a simple GUI.

CLI: If you prefer scripting, automation, or advanced control, the command-line interface lets you run training, export models, start chat, or serve APIs directly from the terminal.

Training methods supported by llama-factory

Method	What it does	Availability
SFT	Instruction Finetuning	Available
LoRA / QLoRA	Low-VRAM fine-tuning	Available
DPO	Direct Preference Optimization	Available
RLHF (PPO + Reward Model)	Reinforcement learning using reward model scoring	Available
RLAIF	AI-feedback RL	Available
Full Fine-Tune	All weights train	Available

Finetune model supported:

- **LLaMA family (Meta)**
Including all LLaMA, LLaMA-2, LLaMA-3 variants
- **Mistral family**
Including Mixtral MoE models
- **Qwen family**
Qwen-1, Qwen-2, Qwen-VL, Qwen-Coder, etc.
- **Gemma models (Google)**
- **Yi model family** (01.AI — China)
- **Phi model family (Microsoft)**
- **Baichuan** (Baichuan Inc — China)
- **other popular community models**
Examples: ChatGLM(Tsinghua University + Zhipu AI), DeepSeek (DeepSeek AI), OpenBuddy, etc.

Dataset Format supported by llama factory

Factory 3 formats samajhta hai:

Alpaca Format (Instruction Finetuning)

```
[
  {
    "instruction": "What is preference alignment?",
    "input": "",
    "output": "Alignment is making LLM follow human preferences..."
  }
]
```

- Columns:
- instruction
 - input
 - output

ShareGPT Format (Chat finetuning)

Used for role-based chat data.

```
[
  {
    "conversations": [
      {"from": "user", "value": "Hello!"},
      {"from": "assistant", "value": "Hi, how can I help you?"}
    ]
  }
]
```

DPO Format (Preference Training)

```
[
  {
    "prompt": "Explain RLHF.",
    "chosen": "RLHF aligns model with human feedback.",
    "rejected": "I don't know about RLHF."
  }
]
```

]

Step-by-Step Finetuning (Web UI)

Base Model Select
Dataset Load
Training Configuration
GPU Optimization
Start Training
Evaluation
Chat With Your Model
Export Model

CLI Training (YAML-Based)

Command	Purpose / Description
llamafactory-cli train config.yaml	Run a training/fine-tuning session (LoRA, QLoRA, Full FT)
llamafactory-cli chat config.yaml	Launch CLI chat after training/export
llamafactory-cli eval config.yaml	Evaluate model (perplexity/custom eval)
llamafactory-cli export config.yaml	Merge adapters and export final model
llamafactory-cli api config_api.yaml	Start an API server endpoint (OpenAI-style)
llamafactory-cli webui	Launch graphical interface (training, eval, chat, export)
llamafactory-cli webchat	Launch web-based chat UI
llamafactory-cli version	Show installed version

YAML Config for QLoRA Training

File: train_llama3_qlora.yaml
model_name_or_path: meta-llama/Meta-Llama-3-8B-Instruct
template: llama3

finetuning_type: lora
dataset: custom
dataset_format: alpaca
train_file: data/train.json

output_dir: output/llama3-qlora
cutoff_len: 1024

per_device_train_batch_size: 1
gradient_accumulation_steps: 8
num_train_epochs: 3

learning_rate: 2e-4
lr_scheduler_type: cosine
warmup_ratio: 0.03

lora_r: 64
lora_alpha: 16
lora_dropout: 0.05

quantization_bit: 4
bf16: true
gradient_checkpointing: true
report_to: none

Run Training

llamafactory-cli train train_llama3_qlora.yaml

Inference with the Trained Model

```
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
```

```
base = "meta-llama/Meta-Llama-3-8B-Instruct"
adapter = "output/llama3-qlora"
```

```
tok = AutoTokenizer.from_pretrained(base)
model = AutoModelForCausalLM.from_pretrained(base)
model = PeftModel.from_pretrained(model, adapter)
```

```
prompt = "Explain Lora in simple terms."
inputs = tok(prompt, return_tensors="pt")
```

```
outputs = model.generate(**inputs, max_new_tokens=200)
print(tok.decode(outputs[0], skip_special_tokens=True))
```

Dataset which I will use to model finetune: <https://huggingface.co/datasets/harpomaxx/unix-commands>

Notebook train_gemma_qlora.yaml X

```
1 %%writefile train_gemma_qlora.yaml
2 model_name_or_path: google/gemma-1.1-2b-it
3 template: gemma
4
5 finetuning_type: lora
6 dataset: custom
7 dataset_format: alpaca
8 train_file: data/train.json
9
10 output_dir: output/gemma-qlora
11 cutoff_len: 2048
12
13 per_device_train_batch_size: 1
14 gradient_accumulation_steps: 8
15 num_train_epochs: 1
16
17 learning_rate: 5e-5
18 lr_scheduler_type: cosine
19 warmup_ratio: 0.03
20
21 lora_r: 64
22 lora_alpha: 16
23 lora_dropout: 0.05
24
25 quantization_bit: 4
26 bf16: false # COLAB T4 DOES NOT SUPPORT BF16 !
27 fp16: true
28
29 gradient_checkpointing: true
30 report_to: none
31
32
```

LLAMAFACTORY_PARAMETER

04 December 2025 22:31

HUB NAME (Model Download Source)

1) huggingface

Model HF Hub se download hota hai — **default recommended source.**

2) modelscope

China-based model hub — kuch Chinese models yaha hote hain.

3) openmind

Local folder ya custom storage se **offline model load** karne ka option.

FINETUNING METHOD

1) LoRA

Model ke sirf small adapters train hote — **fast, low-VRAM, best practical method.**

2) Full

Pure model ke saare weights train hote — **maximum accuracy, extremely heavy.**

3) Freeze

Kuch layers freeze, kuch train — **medium VRAM + stable training.**

4) OFT (Orthogonal Fine-Tuning)

LoRA ka alternative method — **better stability for some models.**

QUANTIZATION BIT

1) none

No quantization — model full precision me load hota — **highest VRAM usage.**

2) 8-bit

Model 8-bit me load — VRAM kam lagti — training normal.

3) 4-bit (QLoRA)

Model 4-bit me load — **minimum VRAM requirement, best choice for FT.**

QUANTIZATION METHOD

1) bnb (BitsAndBytes)

Stable & most used quantization backend — **recommended default.**

2) hqq (High Quality Quantization)

Better accuracy preservation — **slightly heavier than bnb.**

3) etqq

Experimental fast quantization — **not stable for all models.**

CHAT TEMPLATE

chatglm3

ChatGLM-3 model ke sahi conversation formatting ke liye template.

RoPE SCALING (Context Window Upgrade Method)

1) none

No scaling — original context length hi use hoga.

2) linear

Simple linear scaling — **basic long-context extension.**

3) dynamic

Adaptive positional scaling — **more stable for longer context.**

4) yarn

SOTA RoPE method — **best quality for 8K → 32K → 128K upgrades.**

5) llama3

Specifically LLaMA 3 positional embedding scaling ka mode.

STER (Training Speed Enhancer)

1) auto

LLaMA Factory automatically fastest kernel choose karta hai.

2) flash attention

GPU memory aur speed huge improve — **very fast attention kernel.**

3) unsloth

Training ko ultra-fast banata & VRAM half kar deta — **best for laptops/Colab.**

4) liger kernel

Advanced fused kernels — **stable + faster training.**

Stage (Training Type)

Supervised Fine-Tuning (SFT)

Model ko instruction–output pairs se train karta hai.

General instruction tuning ke liye best.

Reward Modeling

Model ko "good vs bad" answers score karna sikhata hai.

RLHF pipeline ka first step.

PPO (Proximal Policy Optimization)

Reinforcement Learning step jisme reward model ke signals se LLM ko optimize karte hain.

ChatGPT-style alignment.

DPO (Direct Preference Optimization)

PPO ka simplified version — **reward model ki zaroorat nahi.**

Chosen vs Rejected answer se alignment.

KTO (Kahneman-Tversky Optimization)

DPO ka improved variant for stable preference alignment.

Low-data preference FT.

Pretraining

Model ko next-token prediction task par train karta hai (raw text).

Agar tu apna model scratch se banana chahe.

Data dir

Your dataset folder location.

Wohi folder jahan train.json ya dataset files rakhe ho.

Dataset

Specific dataset name/file you want to use.

Like train.json, mydata.json, etc.

Learning rate

Model ki weight update speed.

Lower = stable training, Higher = fast par risk.

QLoRA ke liye **5e-5 to 2e-4** best.

Epochs

Dataset kitni baar model ko dikhaya jayega.

Zyada epochs = zyada learning but overfit risk.

Maximum gradient norm (Grad Clip)

Gradient explode hone se bachata hai.

Common safe value = 1.0

Max samples

Dataset ka maximum number of samples use hongi.

Training speed / cost control karne ke liye limit.

Compute type

Model ko kis precision me train karna hai:

bf16

Fast, low-memory, stable.

Best option (A100, H100, 4090 etc.)

fp16

Half precision, memory efficient.

Normal GPUs par safe.

fp32

Full precision.

Very slow + high VRAM usage.

pure_bf16

All operations bf16 me — even more stable for big models.

For premium GPUs only.

Cutoff length

Max input token length (context window during training).

Training time, VRAM usage, and memory directly affect.

Common = **2048 or 4096**

Batch size

Kitne samples ek forward-pass me jayenge.

Low VRAM GPUs par typical = 1 or 2

Gradient accumulation

Batch ko VRAM me fit karne ka trick:

$\text{batch_size} * \text{grad_accumulation} = \text{effective_batch_size}$

Small GPU ko big batch jaisa behave karane ka tareeka.

Val size (%)

Dataset ka kitna percent **validation** ke liye use hoga.

Model quality measure karne ke liye.

0 = no validation.

LR scheduler

Learning rate ka shape / curve.

cosine

Sabse smooth, stable learning curve.

Most recommended.

linear

Linearly LR reduce hota.

constant

LR same rehta.

Scheduler model ko **overfitting / unstable gradients** se bachata hai.

EXTRA CONFIGURATIONS (Training Behavior Tweaks)

Logging steps

Kitne steps ke baad training log print hoga.

Save steps

Kitne steps ke baad model checkpoint save hoga.

Warmup steps

Training ke starting me learning rate ko dheere-dheere increase karne ka phase.

NEFTune alpha

Embeddings me thoda noise add karke generalization improve karta hai.

Extra arguments

JSON format me trainer ko additional advanced parameters dene ka option.

Pack sequences

Multiple small samples ko ek long sample me combine karke training VRAM optimize karta hai.

Use neat packing

Sequence boundaries ko clean rakhta hai taaki attention cross-sequence na ho.

Train on prompt

Prompt ke tokens par bhi loss calculate kare (SFT me rarely used).

Mask history

Chat history ke pehle turns ko loss calculation se exclude karta hai.

Resize token embeddings

Tokenizer vocab badne par embedding layer resize kare.

Enable LLaMA Pro

LLaMA Pro models ke special blocks ko trainable banata hai.

Enable thinking

Reasoning model ke liye extra training mode activate karta hai.

Enable external logger

Loss curves ko WandB / Tensorboard / MLFlow me track karne ka option.

FREEZE TUNING CONFIGURATIONS

Trainable layers

Last X layers ko trainable banata, baaki freeze — memory bachti hai.

Trainable modules

Specific module names ko trainable banata (e.g., q_proj, k_proj).

Extra modules

Additional custom modules specify kar sakte ho training ke liye.

LoRA CONFIGURATIONS

LoRA rank (r)

Adapter matrix size — **higher r = more learning capacity.**

LoRA alpha

Scaling factor for LoRA updates — affects training stability.

LoRA dropout

LoRA layers par dropout lagta — regularization improve hota.

LoRA+ LR Ratio

LoRA B matrices ke learning rate ka ratio.

Create new adapter

Existing adapter overwrite na karke naya adapter create karta hai.

Use rslora

r-adjusted LoRA method for better stability.

Use DoRA

Weight-decomposed LoRA — higher quality FT method.

Use PiSSA

LoRA ka mathematically improved variant (more stable).

LoRA modules

Specify exactly kaunse model modules par LoRA apply karna hai.

Additional modules

Extra modules jinko LoRA apply karna chahte ho.

RLHF CONFIGURATIONS

Beta value

DPO / PPO loss ke temperature jaisa effect — sensitivity control.

Ftx gamma

SFT loss ko PPO/DPO final loss me kitna weight dena hai.

Loss type (sigmoid, margin, hinge)

DPO loss ka mathematical shape choose karta hai.

Reward model

PPO me kaunsa reward model use karoge.

Score norm

Reward scores ko normalize karta hai.

Whiten rewards

Reward signals ka mean remove + variance normalize karta hai — PPO stable hota.

MULTIMODAL CONFIGURATIONS

Freeze vision tower

Image encoder ko freeze karta — only language part train hota.

Freeze multi-modal projector

Vision→text mapping layer ko freeze karta.

Freeze language model

Core LLM ko freeze karke sirf projector train karta.

Image max pixels

Max allowed resolution per image.

Image min pixels

Minimum image size.

Video max pixels

Video frames per max resolution.

Video min pixels

Minimum video resolution.

GaLore CONFIGURATIONS (Memory Optimization)

Use GaLore

Gradients ko low-rank form me compress karta — VRAM huge save hoti.

GaLore rank

Low-rank gradient size.

Update interval

Kitne steps baad GaLore projection update hoga.

GaLore scale

Scaling factor for GaLore gradient updates.

GaLore modules

Modules jahan GaLore apply hoga.

APOLLO CONFIGURATIONS (Advanced Optimizer)

Use APOLLO

Optimizer that improves stability for large LLMs.

APOLLO rank

Optimizer ka low-rank update dimension.

Update interval

Optimizer projection update frequency.

APOLLO scale

Optimizer update scaling.

APOLLO modules

Modules jahan APOLLO apply karoge.

BAdam CONFIGURATIONS (Better Adam)

Enable BAdam

Adam optimizer ka improved version activate karta.

BAdam mode (layer/ratio)

Layer-wise ya ratio-based update select karta.

Switch mode (ascending/descending/random)

Layers ka update order control karta.

Switch interval

Kitne steps baad layer switch hoga.

Update ratio

BAdam update ka scale factor.

SWANLAB CONFIGURATIONS (Experiment Tracking)

Use SwanLab

Training metrics cloud me visualize karne ke liye.

Project name

SwanLab project ka naam.

Experiment name

Specific run ka naam.

Workspace

Cloud workspace.

SwanLab API key

Cloud tracking ke liye authentication.

Mode (cloud / offline)

Online logging or offline logging.

FINAL TRAINING SECTION

Output dir

Final trained model save hone ka folder.

Config path

YAML config save karne ka path.

Device count

Kitne GPUs use karne hai.

DeepSpeed stage

Memory-saver distributed training levels.

Enable offload

Model weights ko CPU me offload karke GPU memory bachata.

Device memory

Current VRAM usage ka live meter.

Epochs vs step

09 December 2025 13:47

You're seeing **many loss values** even though you've set only `num_train_epochs: 3` because of how **logging** works in training:

Here's what's happening:

In your YAML config:

```
logging_steps: 10
```

This means:

Log training metrics every 10 steps (batches) — not per epoch.

What's a "step"?

- 1 **step** = 1 **mini-batch** processed
- Each epoch consists of **many steps** depending on dataset size and batch size

So Why Many Losses?

You are training for **3 epochs**, but if your dataset has e.g. **675 training steps per epoch**, and you log every **10 steps**, you'll get roughly:

$3 \text{ epochs} \times (675 / 10) \approx \sim 200 \text{ logs}$

Each log line (like `{'loss': 1.4, 'epoch': 0.35}`) tells you the **intermediate loss** at that point in training — helpful to monitor learning progress.

Summary

- ☒ Multiple losses = logs every few steps
- ☒ You're seeing good live feedback (not extra epochs)
- ☒ Final saved model/adapters will come after the 3rd epoch if `save_strategy: epoch` is used

Let me know if you'd like to plot these losses into a graph, or switch to logging only per epoch.

You said:

diffecen between ephocs and logging step

ChatGPT said:

Great — let's break it down clearly:

Epoch vs Logging Step: What's the Difference?

☒ Epoch

- **Definition:** One complete pass over the **entire training dataset**.
- Example: If you have 1,000 samples and your batch size is 100 →
→ 1 epoch = 10 steps (10 mini-batches)
- You usually set this with:

```
num_train_epochs: 3
```

→ This means the model will see the whole dataset **3 times**.

☑ Logging Step

- **Definition:** The frequency (in number of training steps) at which you want to **log** training progress like loss, learning rate, etc.
- Set with:

`logging_steps: 10`

→ This means: **every 10 batches**, output the current training loss and metrics.

🔄 Example Together

Let's say:

- Your dataset = 1,000 examples
- `per_device_train_batch_size: 100`
- So, 1 epoch = 10 steps
- `num_train_epochs: 3` → 30 total steps
- `logging_steps: 5` → You log on step 5, 10, 15, 20, 25, 30

You'll see 6 logs total.